FerreStock

Prompts de ingeniería usados para construir el sistema, de la documentación al despliegue

Nombre	FerreStock_prompts_de_ingenieria.pdf
Descripción	Prompts usados con un asistente de IA en cada fase del proyecto (análisis, diseño, preparación, implementación por incrementos, pruebas, CI/CD con Taiga, métricas y monitoreo), con su objetivo, entregables y verificación.
Fecha de creación	2026-10-15
Fecha de actualización	2026-10-15
Autores	Ana Sofía Nagles García · María Mercedes Rincón Valencia · Juan David Camacho Tapiero
Versión	1.0.0
Sistema en línea	https://ferrestock-wtrq.onrender.com

Cómo se usaron los prompts

Cada prompt sigue la misma estructura:

- **Contexto:** el prompt 01 se envía al inicio de cada sesión.
- **Tarea** concreta.
- **Restricciones** del dominio.
- **Entregables** con nombre de archivo.
- **Criterio de verificación.**

Las reglas de trabajo con el asistente fueron:

- **Trabajo por fases con aprobación:** la IA no avanza sin la aprobación del equipo. Desde el incremento 6 se aprobó un "modo bloque" para acelerar.
- **Nada sin evidencia:** todo resultado se reporta con las pruebas ejecutadas y sus números reales; lo que no se pudo ejecutar se declara.
- **Las ambigüedades se registran:** si una regla no está en los documentos, se anota como decisión pendiente (DP) con una propuesta.
- **Seguridad de credenciales:** las claves nunca se escriben en el chat; van como secretos de GitHub o se escriben en la terminal sin mostrarse.
- **Revisión humana:** el equipo revisó y aprobó cada fase; la IA propuso, implementó y verificó.

Índice

1. Contexto — Prompt de contexto (se envía al inicio de cada sesión)
2. Fase 1 · Análisis — Fase 1 — Visión, alcance e interesados
3. Fase 1 · Análisis — Fase 1 — Requisitos, reglas de negocio y casos de uso
4. Fase 1 · Análisis — Fase 1 — Escenarios de aceptación, riesgos y trazabilidad
5. Fase 2 · Diseño — Fase 2 — Arquitectura y decisiones técnicas
6. Fase 2 · Diseño — Fase 2 — Modelo de datos e inventario/costos
7. Fase 2 · Diseño — Fase 2 — API REST, seguridad y frontend
8. Fase 2 · Diseño — Fase 2 — Repositorios, pruebas y entrega
9. Fase 3 · Preparación — Fase 3 — Preparación de los repositorios
10. Fase 4 · Implementación — Fase 4 — Incremento 1: Autenticación
11. Fase 4 · Implementación — Fase 4 — Incremento 2: Gestión de usuarios
12. Fase 4 · Implementación — Fase 4 — Incremento 3: Catálogo: categorías, productos y precios
13. Fase 4 · Implementación — Fase 4 — Incremento 4: Clientes y proveedores
14. Fase 4 · Implementación — Fase 4 — Incremento 5: Núcleo de inventario
15. Fase 4 · Implementación — Fase 4 — Incremento 6: Ajustes e inventario inicial
16. Fase 4 · Implementación — Fase 4 — Incremento 7: Compras y su anulación
17. Fase 4 · Implementación — Fase 4 — Incremento 8: Ventas con pago en efectivo
18. Fase 4 · Implementación — Fase 4 — Incremento 9: Devoluciones
19. Fase 4 · Implementación — Fase 4 — Incremento 10: Comprobantes
20. Fase 4 · Implementación — Fase 4 — Incremento 11: Auditoría y valorización
21. Fase 4 · Implementación — Fase 4 — Modo bloque (incrementos 6 a 9)
22. Fase 5 · Pruebas — Fase 5 — Pruebas
23. Fase 6 · CI/CD — Fase 6 — CI/CD con Taiga y despliegue gratuito
24. Fase 7 · Métricas — Fase 7 — Métricas y monitoreo
25. Transversal — Prompts de verificación y corrección (usados en todas las fases)

01 Contexto

Prompt de contexto (se envía al inicio de cada sesión)

Objetivo: Dar a la IA un marco estable para que todas las respuestas respeten las mismas reglas, sin repetirlas en cada prompt.

```
Actúa como un equipo de ingeniería de software (analista, arquitecto, desarrollador full stack y QA) que acompaña a tres estudiantes de ingeniería en un proyecto académico.

Proyecto: FerreStock, sistema web de inventario para una ferretería en Colombia.
Autores (van en el encabezado de cada archivo): Ana Sofía Nagles García, María Mercedes Rincón Valencia y Juan David Camacho Tapiero.

Reglas que aplican a todas las fases:
1. Trabajamos por fases. No avances a la siguiente sin nuestra aprobación explícita.
2. Escribe en español claro y técnico, con nombres propios del dominio (producto, existencias, kardex, consecutivo). No copies textos de otros proyectos ni de los enunciados.
3. Todo archivo lleva un encabezado con: nombre, descripción, fecha de creación, fecha de actualización, autores y versión.
4. Si un requisito es ambiguo, no inventes la regla: regístrala como "decisión pendiente" (DP-xx) con una propuesta marcada [P].
5. Cada afirmación de "funciona" debe venir acompañada de la prueba que lo demuestra, con números reales.
6. Stack fijo: Java 21, Spring Boot 3, JPA, Lombok y PostgreSQL en el backend; React, Material UI, Formik y Yup en el frontend; GitHub y Taiga para la gestión.
```

Entregables	Ninguno: solo fija el contexto.
Cómo se verificó	La IA resume las reglas antes de empezar y pregunta lo que falte.

02 Fase 1 · Análisis

Fase 1 — Visión, alcance e interesados

Objetivo: Delimitar qué problema resuelve el sistema, para quién y qué queda fuera.

```
Con base en la descripción del negocio (ferretería que vende al detal, compra a proveedores, recibe devoluciones y necesita inventario confiable), redacta:

1. 01_vision_y_alcance.md: problema, objetivos medibles (OBJ-01…), alcance incluido y excluido (por ejemplo, la facturación electrónica real queda fuera y se simula), supuestos y restricciones (plan gratuito, equipo de 3 personas).
2. 02_interesados_y_roles.md: interesados con sus intereses y los dos roles del sistema (Administrador y Usuario operativo), con sus necesidades.

Usa tablas. Termina con una lista de preguntas para el cliente.
```

Entregables	01_vision_y_alcance.md, 02_interesados_y_roles.md y una lista de preguntas abiertas.
Cómo se verificó	Revisión del equipo: el alcance excluido quedó explícito y las preguntas se convirtieron en DP-01 a DP-22.

03 Fase 1 · Análisis

Fase 1 — Requisitos, reglas de negocio y casos de uso

Objetivo: Obtener requisitos verificables y trazables, sin ambigüedades ocultas.

```
Redacta 03_requisitos.md con requisitos funcionales agrupados por módulo, con códigos RF-<MÓDULO>-NNN (AUT, PRO, PRE, INV, COS, COM, VEN, PAG, TER, DEV, AJU, DOC, AUD, MON). Cada fila trae: descripción, criterio de aceptación verificable, rol, prioridad MoSCoW, regla de negocio relacionada y escenario que lo prueba.

Agrega los no funcionales (RNF-SEG, RNF-USA, RNF-FIA, RNF-DES, RNF-ADF…) con su forma de verificación.

Luego escribe 04_reglas_del_negocio.md (RN-01…), 05_casos_de_uso.md (actor, flujo principal y errores) y 06_permisos_por_rol.md (matriz de permisos con el principio de mínimo privilegio).

Restricciones:
- Las existencias nunca pueden quedar negativas, ni siquiera con operaciones simultáneas.
- El costo es promedio ponderado (CPP), con aritmética decimal exacta.
- Los documentos no se borran: se anulan.
```

Entregables	Cuatro documentos en Markdown con tablas y códigos estables.
Cómo se verificó	Cada RF tiene su criterio de aceptación; ningún criterio usa palabras vagas como "rápido" o "fácil".

04 Fase 1 · Análisis

Fase 1 — Escenarios de aceptación, riesgos y trazabilidad

Objetivo: Convertir los requisitos en escenarios comprobables y dejar visibles los riesgos y las decisiones pendientes.

```
Escribe 07_escenarios_de_aceptacion.md en formato Dado / Cuando / Entonces, con códigos CA-<MÓDULO>-NN y números concretos. Por ejemplo: "Dado un producto con 10 unidades a CPP 1.000, cuando se compran 30 a 1.400, entonces el CPP es 1.300".

Incluye escenarios de concurrencia (dos ventas simultáneas de la última unidad), de límites (cantidades con decimales en productos que no se fraccionan) y de seguridad (un rol sin permiso recibe 403).

Después escribe 08_riesgos_y_decisiones.md, con riesgos calificados por probabilidad e impacto y su mitigación, y una tabla de decisiones pendientes. Cierra con 09_trazabilidad.md: objetivo → requisito → caso de uso → escenario → requisito no funcional.
```

Entregables	Tres documentos; la matriz de trazabilidad no deja requisitos huérfanos.
Cómo se verificó	Script de revisión: todo RF aparece en la matriz y todo CA apunta a un RF.

05 Fase 2 · Diseño

Fase 2 — Arquitectura y decisiones técnicas

Objetivo: Elegir la arquitectura con argumentos y dejar cada decisión registrada.

```
Diseña la arquitectura de FerreStock:
- Backend en monolito modular por dominio (autenticacion, usuarios, catalogo, terceros,
inventario, compras, ventas, devoluciones, ajustes, comprobantes, auditoria, monitoreo),
con capas Controller → Service → Repository.
- Frontend SPA en el mismo origen que la API: el hosting reenvía /api al backend.

Entrega 01_arquitectura.md, con diagramas Mermaid de componentes y dependencias, y
09_decisiones_tecnicas.md con ADRs numerados (contexto, decisión, alternativas
descartadas, consecuencias).

Resuelve como propuesta [P] las decisiones pendientes de la Fase 1 que bloqueen el
diseño, por ejemplo:
- cómo se anula una venta;
- a qué costo reingresa una devolución;
- si hay consecutivos para todos los documentos.

Justifica cada propuesta con la integridad del inventario.
```

Entregables	Arquitectura y ADR-01 a ADR-19, más las DP resueltas como propuesta.
Cómo se verificó	El equipo aprobó las propuestas; las DP sin resolver (DP-20) quedaron explícitas.

06 Fase 2 · Diseño

Fase 2 — Modelo de datos e inventario/costos

Objetivo: Un esquema que haga imposible, por diseño, lo que las reglas prohíben.

```
Diseña el modelo de datos en PostgreSQL (02_modelo_de_datos.md y modelo-er.mmd):
- Usa CHECK, UNIQUE y FK para proteger las reglas: existencias >= 0, un producto por
línea de documento y motivo de anulación obligatorio si el estado es ANULADA.
- Haz el kardex y la auditoría inmutables con un trigger que rechace UPDATE y DELETE.
- Usa consecutivos en una tabla con bloqueo de fila (sin huecos ante un rollback).

En 03_inventario_y_costos.md:
- Escribe la tabla de efectos de cada operación sobre existencias y CPP, y las fórmulas
con precisión (cantidad 3 decimales, costo 4, dinero 2, HALF_UP).
- Define las reglas de anulación: cuándo se permite, cuándo se bloquea y por qué.
- Define los controles de las devoluciones y la consulta de verificación de consistencia
entre productos y kardex.
```

Entregables	Modelo de datos, diagrama ER y reglas de costos con ejemplos numéricos.
Cómo se verificó	Los ejemplos numéricos se convirtieron después en pruebas unitarias (CA-COS-01 a 03).

07 Fase 2 · Diseño

Fase 2 — API REST, seguridad y frontend

Objetivo: Contratos claros entre frontend y backend, y una seguridad pensada desde el diseño.

```
Define la API en 04_api_rest.md:
- Recursos versionados (/api/v1), paginación, filtros y errores en formato ProblemDetail
(RFC 9457) con un código estable por caso (EXISTENCIAS_INSUFICIENTES,
VERSION_DESACTUALIZADA…) y errores por campo.
- Los montos viajan como texto para no perder precisión.
- El precio de venta nunca viaja desde el cliente.

En 05_seguridad.md cubre:
- Sesión en el servidor con cookie HttpOnly.
- "Recordarme" con token persistente.
- CSRF con cookie y cabecera.
- Bloqueo por intentos fallidos.
- @PreAuthorize por método.
- Revalidación del estado de la cuenta en cada petición.

En 06_frontend.md define la estructura por módulos, los componentes compartidos, la
validación con Yup alineada con el backend, i18n (es/en), accesibilidad (WCAG 2.1 AA) y
diseño responsive.

En 07_flujos_criticos.md agrega diagramas de secuencia de la venta concurrente, la
compra, la devolución, el ajuste y la anulación.
```

Entregables	Cuatro documentos de diseño listos para implementar.
Cómo se verificó	Revisión cruzada: cada endpoint corresponde a un RF y cada RF de la API tiene su permiso por rol.

08 Fase 2 · Diseño

Fase 2 — Repositorios, pruebas y entrega

Objetivo: Decidir desde el diseño cómo se versiona, se prueba y se despliega.

```
Escribe 08_repositorios_y_entrega.md con:
- Repositorios separados para backend, frontend y documentación.
- Ramas main, develop y feature/*, con commits Conventional Commits en español.
- Formato obligatorio de encabezados y su verificación automática.
- Estrategia de pruebas por tipo: unitarias, integración con Testcontainers, seguridad,
mutación con PIT, E2E con Playwright y axe, compatibilidad con Selenium, y carga con k6.
- Diseño del pipeline: en cada push corren las pruebas; si fallan, se crea un issue y una
tarea en Taiga asignados al autor del commit; si vuelven a pasar, se cierran.
```

Entregables	Documento de entrega y estrategia de pruebas.
Cómo se verificó	Sirvió de guion para las fases 5 y 6.

09 Fase 3 · Preparación

Fase 3 — Preparación de los repositorios

Objetivo: Una base técnica que compile, pruebe y verifique encabezados antes de escribir funcionalidad.

```
Prepara los dos repositorios de código.

Backend (Spring Boot 3.5, Java 21, Maven Wrapper):
- Paquetes por módulo.
- Configuración de seguridad (sesión, CSRF con cookie, CORS cerrado).
- Manejador global de errores con ProblemDetail y correlación.
- i18n con MessageSource.
- OpenAPI solo en desarrollo.
- Jackson con BigDecimal como texto.
- Perfiles dev, test y prod.
- Scripts db/01_esquema.sql, db/02_datos_iniciales.sql y crear_bd.sh (crea el rol de
aplicación y el admin con clave temporal).
- Scripts compilar, probar, mutacion, arrancar y verificar_encabezados.
- Pruebas de la base técnica con Testcontainers.

Frontend (Vite, React, MUI):
- Tokens de diseño y tema claro/oscuro.
- Cliente Axios con CSRF y Accept-Language.
- Rutas protegidas por sesión y rol con carga diferida.
- Plantilla responsive y accesible, i18n por namespaces.
- Playwright con tres proyectos (Chromium, Firefox, móvil) y una API simulada.

No implementes funcionalidad de negocio todavía.
```

Entregables	Dos repositorios con su base técnica en verde.
Cómo se verificó	./mvnw verify en verde; lint, i18n, build y E2E de la plantilla en verde; informe 01_informe_preparacion.md.

10 Fase 4 · Implementación

Fase 4 — Incremento 1: Autenticación

Objetivo: Implementar autenticación de punta a punta, con pruebas, sin romper lo anterior.

```
Implementa el incremento 1 (Autenticación) en ramas feature/ de ambos repositorios,
siguiendo el diseño aprobado.

Alcance (RF-AUT-001 a 005, 007 y 008; RF-AUD-001): inicio de sesión con "Recordarme" (7
días), cierre de sesión, bloqueo tras 5 intentos fallidos por 15 minutos, cambio
obligatorio de la contraseña temporal, auditoría de inicios exitosos y fallidos, y el
menú según el rol.

Backend: entidad, repositorio, servicio, controlador, DTO validados con mensajes i18n,
códigos de error estables, auditoría, Javadoc y encabezados.
Frontend: servicio, pantallas con MUI (tabla en escritorio y tarjetas en móvil),
formulario con Formik y Yup con las mismas reglas que el backend, i18n es/en, JSDoc y
accesibilidad.

Pruebas obligatorias:
- Integración con PostgreSQL real que cubra CA-AUT-01 a 06 y 08.
- Unitarias de las reglas.
- E2E con Playwright (incluida una revisión con axe).

Cuida especialmente: Que el mensaje de credenciales inválidas sea genérico; que una
cuenta desactivada pierda la sesión en la siguiente petición.

Al terminar, corre la batería completa. Escribe el informe del incremento con los
resultados reales y los defectos encontrados, y espera nuestra aprobación para fusionar
en develop.
```

Entregables	Código, pruebas e informe del incremento 1.
Cómo se verificó	Backend completo en verde (Testcontainers), E2E en verde y, desde el incremento 6, una verificación manual con la base real.

11 Fase 4 · Implementación

Fase 4 — Incremento 2: Gestión de usuarios

Objetivo: Implementar gestión de usuarios de punta a punta, con pruebas, sin romper lo anterior.

```
Implementa el incremento 2 (Gestión de usuarios) en ramas feature/ de ambos repositorios,
siguiendo el diseño aprobado.

Alcance (RF-AUT-006): alta, edición, cambio de rol, desactivación y restablecimiento de
contraseña, solo para el ADMIN; un administrador no puede desactivarse ni quitarse el rol
a sí mismo; al cambiar el rol o la contraseña se cierran las sesiones activas.

Backend: entidad, repositorio, servicio, controlador, DTO validados con mensajes i18n,
códigos de error estables, auditoría, Javadoc y encabezados.
Frontend: servicio, pantallas con MUI (tabla en escritorio y tarjetas en móvil),
formulario con Formik y Yup con las mismas reglas que el backend, i18n es/en, JSDoc y
accesibilidad.

Pruebas obligatorias:
- Integración con PostgreSQL real que cubra CA-AUT-07.
- Unitarias de las reglas.
- E2E con Playwright (incluida una revisión con axe).

Cuida especialmente: Que la sesión de un usuario desactivado se invalide de inmediato
(revalidación por petición, ADR-19).

Al terminar, corre la batería completa. Escribe el informe del incremento con los
resultados reales y los defectos encontrados, y espera nuestra aprobación para fusionar
en develop.
```

Entregables	Código, pruebas e informe del incremento 2.
Cómo se verificó	Backend completo en verde (Testcontainers), E2E en verde y, desde el incremento 6, una verificación manual con la base real.

12 Fase 4 · Implementación

Fase 4 — Incremento 3: Catálogo: categorías, productos y precios

Objetivo: Implementar catálogo: categorías, productos y precios de punta a punta, con pruebas, sin romper lo anterior.

```
Implementa el incremento 3 (Catálogo: categorías, productos y precios) en ramas feature/
de ambos repositorios, siguiendo el diseño aprobado.

Alcance (RF-PRO, RF-PRE, RF-COS-002): código único sin distinguir mayúsculas, bloqueo
optimista con versión, desactivación en lugar de borrado, cambio de precio auditado y
costo visible solo para el ADMIN.

Backend: entidad, repositorio, servicio, controlador, DTO validados con mensajes i18n,
códigos de error estables, auditoría, Javadoc y encabezados.
Frontend: servicio, pantallas con MUI (tabla en escritorio y tarjetas en móvil),
formulario con Formik y Yup con las mismas reglas que el backend, i18n es/en, JSDoc y
accesibilidad.

Pruebas obligatorias:
- Integración con PostgreSQL real que cubra CA-PRO-01 a 06, 08, 10; CA-PRE-01, 02.
- Unitarias de las reglas.
- E2E con Playwright (incluida una revisión con axe).

Cuida especialmente: Que dos ediciones simultáneas devuelvan 409 VERSION_DESACTUALIZADA y
la pantalla ofrezca recargar.

Al terminar, corre la batería completa. Escribe el informe del incremento con los
resultados reales y los defectos encontrados, y espera nuestra aprobación para fusionar
en develop.
```

Entregables	Código, pruebas e informe del incremento 3.
Cómo se verificó	Backend completo en verde (Testcontainers), E2E en verde y, desde el incremento 6, una verificación manual con la base real.

13 Fase 4 · Implementación

Fase 4 — Incremento 4: Clientes y proveedores

Objetivo: Implementar clientes y proveedores de punta a punta, con pruebas, sin romper lo anterior.

```
Implementa el incremento 4 (Clientes y proveedores) en ramas feature/ de ambos
repositorios, siguiendo el diseño aprobado.

Alcance (RF-TER): un solo registro con los roles cliente y proveedor, validación del
dígito de verificación del NIT con el algoritmo módulo 11, persona jurídica solo con NIT
y documento único.

Backend: entidad, repositorio, servicio, controlador, DTO validados con mensajes i18n,
códigos de error estables, auditoría, Javadoc y encabezados.
Frontend: servicio, pantallas con MUI (tabla en escritorio y tarjetas en móvil),
formulario con Formik y Yup con las mismas reglas que el backend, i18n es/en, JSDoc y
accesibilidad.

Pruebas obligatorias:
- Integración con PostgreSQL real que cubra CA-TER-01 a 04.
- Unitarias de las reglas.
- E2E con Playwright (incluida una revisión con axe).

Cuida especialmente: Que el DV inválido se marque en su campo, tanto en Yup como desde el
backend.

Al terminar, corre la batería completa. Escribe el informe del incremento con los
resultados reales y los defectos encontrados, y espera nuestra aprobación para fusionar
en develop.
```

Entregables	Código, pruebas e informe del incremento 4.
Cómo se verificó	Backend completo en verde (Testcontainers), E2E en verde y, desde el incremento 6, una verificación manual con la base real.

14 Fase 4 · Implementación

Fase 4 — Incremento 5: Núcleo de inventario

Objetivo: Implementar núcleo de inventario de punta a punta, con pruebas, sin romper lo anterior.

```
Implementa el incremento 5 (Núcleo de inventario) en ramas feature/ de ambos
repositorios, siguiendo el diseño aprobado.

Alcance (RF-INV, RF-COS-001, RF-DOC-001): un servicio único que mueve existencias y CPP
(propagación MANDATORY), bloqueo FOR UPDATE de los productos en orden de id, kardex
inmutable con saldo y CPP anterior y resultante, consecutivos sin huecos, existencias,
kardex filtrable, valorización y verificación de consistencia.

Backend: entidad, repositorio, servicio, controlador, DTO validados con mensajes i18n,
códigos de error estables, auditoría, Javadoc y encabezados.
Frontend: servicio, pantallas con MUI (tabla en escritorio y tarjetas en móvil),
formulario con Formik y Yup con las mismas reglas que el backend, i18n es/en, JSDoc y
accesibilidad.

Pruebas obligatorias:
- Integración con PostgreSQL real que cubra CA-INV-01, 02 y la concurrencia real.
- Unitarias de las reglas.
- E2E con Playwright (incluida una revisión con axe).

Cuida especialmente: Que 2 ventas simultáneas de la última unidad confirmen exactamente 1
y que 50 consecutivos simultáneos salgan distintos y seguidos.

Al terminar, corre la batería completa. Escribe el informe del incremento con los
resultados reales y los defectos encontrados, y espera nuestra aprobación para fusionar
en develop.
```

Entregables	Código, pruebas e informe del incremento 5.
Cómo se verificó	Backend completo en verde (Testcontainers), E2E en verde y, desde el incremento 6, una verificación manual con la base real.

15 Fase 4 · Implementación

Fase 4 — Incremento 6: Ajustes e inventario inicial

Objetivo: Implementar ajustes e inventario inicial de punta a punta, con pruebas, sin romper lo anterior.

```
Implementa el incremento 6 (Ajustes e inventario inicial) en ramas feature/ de ambos repositorios, siguiendo el diseño aprobado.

Alcance (RF-AJU): ajustes de entrada (con costo, ponderan) y de salida (al CPP vigente) solo para el ADMIN, con motivo obligatorio; el inventario inicial es un ajuste de entrada; vista previa del efecto antes de guardar.

Backend: entidad, repositorio, servicio, controlador, DTO validados con mensajes i18n, códigos de error estables, auditoría, Javadoc y encabezados.
Frontend: servicio, pantallas con MUI (tabla en escritorio y tarjetas en móvil), formulario con Formik y Yup con las mismas reglas que el backend, i18n es/en, JSDoc y accesibilidad.

Pruebas obligatorias:
- Integración con PostgreSQL real que cubra CA-AJU-01 a 04.
- Unitarias de las reglas.
- E2E con Playwright (incluida una revisión con axe).

Cuida especialmente: Que una salida mayor que las existencias se rechace sin consumir el consecutivo.

Al terminar, corre la batería completa. Escribe el informe del incremento con los resultados reales y los defectos encontrados, y espera nuestra aprobación para fusionar en develop.
```

Entregables	Código, pruebas e informe del incremento 6.
Cómo se verificó	Backend completo en verde (Testcontainers), E2E en verde y, desde el incremento 6, una verificación manual con la base real.

16 Fase 4 · Implementación

Fase 4 — Incremento 7: Compras y su anulación

Objetivo: Implementar compras y su anulación de punta a punta, con pruebas, sin romper lo anterior.

```
Implementa el incremento 7 (Compras y su anulación) en ramas feature/ de ambos
repositorios, siguiendo el diseño aprobado.

Alcance (RF-COM, RF-COS-001): compra recibida completa en una transacción (si una línea
falla no queda nada y se dice cuál), anulación solo del ADMIN que restaura el CPP
anterior y se bloquea si ya se vendió o hubo entradas posteriores.

Backend: entidad, repositorio, servicio, controlador, DTO validados con mensajes i18n,
códigos de error estables, auditoría, Javadoc y encabezados.
Frontend: servicio, pantallas con MUI (tabla en escritorio y tarjetas en móvil),
formulario con Formik y Yup con las mismas reglas que el backend, i18n es/en, JSDoc y
accesibilidad.

Pruebas obligatorias:
- Integración con PostgreSQL real que cubra CA-COM-01 a 04, CA-COS-01, 02.
- Unitarias de las reglas.
- E2E con Playwright (incluida una revisión con axe).

Cuida especialmente: Que la anulación restaure el CPP exacto y que la verificación de
consistencia quede limpia.

Al terminar, corre la batería completa. Escribe el informe del incremento con los
resultados reales y los defectos encontrados, y espera nuestra aprobación para fusionar
en develop.
```

Entregables	Código, pruebas e informe del incremento 7.
Cómo se verificó	Backend completo en verde (Testcontainers), E2E en verde y, desde el incremento 6, una verificación manual con la base real.

17 Fase 4 · Implementación

Fase 4 — Incremento 8: Ventas con pago en efectivo

Objetivo: Implementar ventas con pago en efectivo de punta a punta, con pruebas, sin romper lo anterior.

```
Implementa el incremento 8 (Ventas con pago en efectivo) en ramas feature/ de ambos repositorios, siguiendo el diseño aprobado.

Alcance (RF-VEN, RF-PAG): venta y pago en una sola operación, precio vigente tomado del servidor, rechazo por existencias con producto y disponibles, pago insuficiente o en otro medio rechazado, cambio calculado y anulación al costo histórico, bloqueada si hay devoluciones.

Backend: entidad, repositorio, servicio, controlador, DTO validados con mensajes i18n, códigos de error estables, auditoría, Javadoc y encabezados.
Frontend: servicio, pantallas con MUI (tabla en escritorio y tarjetas en móvil), formulario con Formik y Yup con las mismas reglas que el backend, i18n es/en, JSDoc y accesibilidad.

Pruebas obligatorias:
- Integración con PostgreSQL real que cubra CA-VEN-01 a 06, CA-PAG-01, 02.
- Unitarias de las reglas.
- E2E con Playwright (incluida una revisión con axe).

Cuida especialmente: Que el costo de la línea quede guardado y solo lo vea el ADMIN.

Al terminar, corre la batería completa. Escribe el informe del incremento con los resultados reales y los defectos encontrados, y espera nuestra aprobación para fusionar en develop.
```

Entregables	Código, pruebas e informe del incremento 8.
Cómo se verificó	Backend completo en verde (Testcontainers), E2E en verde y, desde el incremento 6, una verificación manual con la base real.

18 Fase 4 · Implementación

Fase 4 — Incremento 9: Devoluciones

Objetivo: Implementar devoluciones de punta a punta, con pruebas, sin romper lo anterior.

```
Implementa el incremento 9 (Devoluciones) en ramas feature/ de ambos repositorios,
siguiendo el diseño aprobado.

Alcance (RF-DEV, RF-COS-003): devolución con venta (mismo cliente y sin superar lo
vendido acumulado, al costo histórico) o sin venta (al CPP vigente y auditada para
revisión), con líneas aceptadas o rechazadas y quién validó.

Backend: entidad, repositorio, servicio, controlador, DTO validados con mensajes i18n,
códigos de error estables, auditoría, Javadoc y encabezados.
Frontend: servicio, pantallas con MUI (tabla en escritorio y tarjetas en móvil),
formulario con Formik y Yup con las mismas reglas que el backend, i18n es/en, JSDoc y
accesibilidad.

Pruebas obligatorias:
- Integración con PostgreSQL real que cubra CA-DEV-01 a 03, CA-COS-03.
- Unitarias de las reglas.
- E2E con Playwright (incluida una revisión con axe).

Cuida especialmente: Que CA-COS-03 dé exactamente 1.166,6667.

Al terminar, corre la batería completa. Escribe el informe del incremento con los
resultados reales y los defectos encontrados, y espera nuestra aprobación para fusionar
en develop.
```

Entregables	Código, pruebas e informe del incremento 9.
Cómo se verificó	Backend completo en verde (Testcontainers), E2E en verde y, desde el incremento 6, una verificación manual con la base real.

19 Fase 4 · Implementación

Fase 4 — Incremento 10: Comprobantes

Objetivo: Implementar comprobantes de punta a punta, con pruebas, sin romper lo anterior.

```
Implementa el incremento 10 (Comprobantes) en ramas feature/ de ambos repositorios,
siguiendo el diseño aprobado.

Alcance (RF-DOC-002 a 005): comprobante armado desde el documento (nunca guardado
aparte), con encabezado del negocio, líneas, totales, pago y la leyenda de documento
simulado en el idioma de la petición; búsqueda unificada y vista imprimible.

Backend: entidad, repositorio, servicio, controlador, DTO validados con mensajes i18n,
códigos de error estables, auditoría, Javadoc y encabezados.
Frontend: servicio, pantallas con MUI (tabla en escritorio y tarjetas en móvil),
formulario con Formik y Yup con las mismas reglas que el backend, i18n es/en, JSDoc y
accesibilidad.

Pruebas obligatorias:
- Integración con PostgreSQL real que cubra CA-DOC-01 a 03.
- Unitarias de las reglas.
- E2E con Playwright (incluida una revisión con axe).

Cuida especialmente: Que los comprobantes de ajuste solo los vea el ADMIN.

Al terminar, corre la batería completa. Escribe el informe del incremento con los
resultados reales y los defectos encontrados, y espera nuestra aprobación para fusionar
en develop.
```

Entregables	Código, pruebas e informe del incremento 10.
Cómo se verificó	Backend completo en verde (Testcontainers), E2E en verde y, desde el incremento 6, una verificación manual con la base real.

20 Fase 4 · Implementación

Fase 4 — Incremento 11: Auditoría y valorización

Objetivo: Implementar auditoría y valorización de punta a punta, con pruebas, sin romper lo anterior.

```
Implementa el incremento 11 (Auditoría y valorización) en ramas feature/ de ambos
repositorios, siguiendo el diseño aprobado.

Alcance (RF-AUD-002, 003; RF-COS-002): pantalla de auditoría para el ADMIN con filtros
por usuario, acción, fechas y texto; ninguna operación de edición ni borrado sobre los
eventos.

Backend: entidad, repositorio, servicio, controlador, DTO validados con mensajes i18n,
códigos de error estables, auditoría, Javadoc y encabezados.
Frontend: servicio, pantallas con MUI (tabla en escritorio y tarjetas en móvil),
formulario con Formik y Yup con las mismas reglas que el backend, i18n es/en, JSDoc y
accesibilidad.

Pruebas obligatorias:
- Integración con PostgreSQL real que cubra CA-AUD-01, 02.
- Unitarias de las reglas.
- E2E con Playwright (incluida una revisión con axe).

Cuida especialmente: Que PUT y DELETE sobre la auditoría no existan (4xx).

Al terminar, corre la batería completa. Escribe el informe del incremento con los
resultados reales y los defectos encontrados, y espera nuestra aprobación para fusionar
en develop.
```

Entregables	Código, pruebas e informe del incremento 11.
Cómo se verificó	Backend completo en verde (Testcontainers), E2E en verde y, desde el incremento 6, una verificación manual con la base real.

21 Fase 4 · Implementación

Fase 4 — Modo bloque (incrementos 6 a 9)

Objetivo: Acelerar sin perder control cuando el equipo ya confía en el proceso.

```
Ya aprobamos el proceso. Haz los incrementos 6 a 9 seguidos, sin detenerte a pedir
aprobación entre uno y otro:
- Cada incremento va en su rama y se fusiona en develop cuando pasan el backend completo
y las E2E de Chromium.
- Al final del bloque, corre la matriz E2E completa (Chromium, Firefox y móvil) y haz una
verificación manual en el navegador: inventario inicial → compra → venta → devolución,
revisando kardex, CPP y auditoría.
- Escribe un informe por incremento y un resumen corto del bloque con lo hecho, los
errores y lo pendiente.
- Detente solo ante una decisión de negocio no resuelta o un bloqueo real.
```

Entregables	Cuatro incrementos fusionados, informes y resumen del bloque.
Cómo se verificó	La matriz completa pasó; la verificación manual coincidió con el CPP calculado a mano (450).

22 Fase 5 · Pruebas

Fase 5 — Pruebas

Objetivo: Evidencia de calidad por tipo de prueba, con números reales y el catálogo de casos.

```
Completa la Fase 5:
1. Mutación con PIT sobre los servicios y compartido; reporta el puntaje y la fuerza de las pruebas, y explica las diferencias.
2. Compatibilidad con Selenium WebDriver contra la app real (ingreso, idioma, nueva venta, sesión persistente y ruta protegida); Chrome en local y Edge en CI.
3. Carga, estrés y picos con k6: listados concurrentes, 200 ventas del mismo producto con 50 usuarios (solo 201 o 409, nunca 500, y kardex consistente al final) y una ráfaga de usuarios.
4. Genera el catálogo de casos de prueba desde los nombres reales de las pruebas, clasificado por tipo.
5. Escribe el informe de pruebas. Si algo no se pudo ejecutar, dilo y explica por qué.
```

Entregables	01_catalogo_casos.md (220 casos), 02_informe_pruebas.md, scripts de k6 y pruebas de Selenium.
Cómo se verificó	Selenium 3/3 contra la app real; PIT 25 % (fuerza 83 %), con su explicación; k6 documentado como no ejecutado en el equipo local.

23 Fase 6 · CI/CD

Fase 6 — CI/CD con Taiga y despliegue gratuito

Objetivo: Que cada cambio se pruebe solo, que despliegue si todo pasa y que los fallos se conviertan en trabajo asignado en Taiga.

```
Configura la integración continua y el despliegue:
1. GitHub Actions por repositorio, en push y pull request a develop y main:
   - Backend: encabezados, pruebas y cobertura; en main, también mutación, imagen Docker, despliegue y verificación de salud.
   - Frontend: lint, i18n, build y Playwright; en main, también despliegue y Selenium en Edge.
2. Script taiga_reportar.sh para el proyecto de Taiga indicado:
   - Si una etapa falla, crea un issue y una tarea asignados al autor del commit, o comenta el issue si ya existía.
   - Si la etapa vuelve a pasar, cierra el issue y la tarea.
   - Pruébalo contra un Taiga simulado antes de usar el real.
3. Dockerfiles, nginx como proxy de /api (mismo origen), un blueprint de Render y la inicialización de la base en Neon (planes gratuitos).
4. Scripts de repositorio: nueva_rama, integrar, liberar y compilar_y_probar.

Las credenciales nunca pasan por el chat: van como secretos de GitHub o se escriben en la terminal sin mostrarse.
```

Entregables	Workflows, scripts, Dockerfiles, render.yaml y 01_cicd_y_taiga.md.
Cómo se verificó	El pipeline real encontró dos defectos (nombre de la imagen en GHCR y una región desplazable sin acceso por teclado), que se corrigieron en la v1.0.2; el sistema quedó en línea.

24 Fase 7 · Métricas

Fase 7 — Métricas y monitoreo

Objetivo: Medir el trabajo del equipo y la salud de la aplicación con datos reales.

```
Implementa:
1. Registro de errores 500 con correlación. Si el mismo error ocurre 5 veces en 10
minutos en la misma ruta, crea un solo issue "[Producción]" en Taiga.
2. Endpoints de monitoreo para el ADMIN: errores con su referencia en Taiga y métricas de
operación (ventas del día, anulaciones, productos sin existencias, valor del inventario,
errores, accesos denegados e intentos de ingreso fallidos).
3. Script metricas_taiga.py que lea el proyecto real de Taiga y calcule:
   – avance del tablero;
   – tareas e issues abiertos y cerrados por integrante;
   – horas promedio de resolución;
   – fallos de CI y errores de producción.
4. Script cargar_backlog.py que cree en Taiga una historia por módulo y una tarea por
requisito (simulación por defecto).

Prueba el reporte de errores recurrentes con Taiga simulado.
```

Entregables	Monitoreo en el backend, scripts de métricas y backlog, y 01_metricas_y_monitoreo.md.
Cómo se verificó	MonitoreoIT: 6 errores iguales generan un solo issue; las métricas se leyeron del Taiga real.

25 Transversal

Prompts de verificación y corrección (usados en todas las fases)

Objetivo: Evitar dar por bueno lo que no se probó.

```
• "Antes de decir que está listo, corre la batería completa y dame los números reales
(pasan, fallan, omitidas). Si algo falla, distingue un fallo real de un problema del
entorno o de la prueba, y explícalo."

• "Este error de E2E solo aparece en móvil: reprodúcelo localmente, encuentra la causa
raíz y corrige el código, no la prueba, salvo que la prueba esté mal planteada."

• "Revisa con axe la página y los diálogos abiertos. Si hay violaciones WCAG A/AA,
corrígelas en el componente compartido para que no se repitan en otras pantallas."

• "Haz una verificación manual de punta a punta con la base real y compara el kardex y el
costo promedio con un cálculo hecho a mano."

• "Actualiza el informe con lo que de verdad pasó: qué falló, por qué y cómo se
corrigió."
```

Entregables	Informes con resultados reales y defectos documentados.
Cómo se verificó	Así aparecieron y se corrigieron, entre otros, la prueba dependiente del orden por CSRF, la regla Yup de productos repetidos y el contraste durante animaciones.